



PDI: ATIVIDADE 01 : REGISTRO DE ENTREGA

Queries Complexas e Indexacao no Supabase (PostgreSQL)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 03 : Fullstack e Modelagem de Dados

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

pdi-fullstack-modelagem-dados-a1

1. Contexto

O Supabase que sustenta a operacao SDR IA e a fila `mt_jobs` sofria com queries escritas sem plano de execucao: JOINS pesados sem indice, leituras sequenciais em tabelas de milhoes de linhas, funcoes com RLS no caminho quente das consultas e `autovacuum` desconfigurado. O resultado eram timeouts de webhook, dashboards que abriam em 8s ou mais e fila com backlog invisivel.

Referencias de volumetria V4: `mt_jobs` com 2,4M linhas, `sync_log` com 12M, `events` com 8M particionados, `leads` com 900k e `dashboard_daily` com 60k linhas por mes.

2. Diagnóstico

Com `EXPLAIN (ANALYZE, BUFFERS)` como metodo, os sinais de gargalo aparecem no plano: `Seq Scan` em tabela acima de 100k linhas, `Nested Loop` com re-scan alto por FK sem indice, `Sort` explicito sobre coluna nao indexada, `Hash Join` com spill em temp file e estatisticas obsoletas.

CASO	SINTOMA	CAUSA-RAIZ
Fila <code>mt_jobs</code>	pick de 1,8s com Seq Scan	filtro sem indice composto + LEFT JOIN desnecessario
Sync CRM	JOIN de auditoria em 4,2s	FK sem indice + filtro em coluna sem indice
Dashboard	agregacao em 8,4s sobre 12M linhas	<code>count (DISTINCT)</code> + janela sem materializacao

3. Solução

Indices certos por padrao de acesso, particionamento por range com TTL, autovacuum calibrado e RLS com policies simples. Nenhum indice foi aplicado em producao nesta etapa: apenas documentado e provado.

- **Fila `mt_jobs`:** indice (`queue`, `status`, `scheduled_at`) mais remocao do LEFT JOIN. Plano virou Index Scan e a latencia saiu de 1,8s para 4ms.
- **Sync CRM:** indice em (`client_id`, `object`, `synced_at` DESC) e CTE de janela. Saiu de 4,2s para 180ms.
- **Dashboard:** materializacao parcial (tabela agregada + indice BRIN no tempo). Saiu de 8,4s para 1,1s.

4. Como funciona

Metodo EXPLAIN antes/depois

FLUXO DE OTIMIZACAO

1

Medir com `EXPLAIN (ANALYZE, BUFFERS)`
Plano e fonte da verdade. Procurar Seq Scan, re-scan, Sort e spill.

2

Indice composto `igualdade, range, ORDER BY`
Um B-tree cobre filtro e ordenacao sem Sort. `1,8s para 4ms`

3

Tipo certo: **GIN** e **BRIN**

GIN para arrays/jsonb. BRIN para series temporais, 10x menor. **8,4s para 1,1s**

4

Particionar + **CONCURRENTLY**

Range com TTL e pruning. Aplicar fora de pico sem AccessExclusiveLock.

meta: pick abaixo de 10ms

2-sql/06-planos-antes-depois.md reproduz os 3 casos no staging

● provado

● homologacao

5. Entregas

PASTA	CONTEUDO
1-standards	PERFORMANCE-SUPABASE.md: planos de execucao, EXPLAIN, indices, particionamento, autovacuum
2-sql	5 PoCs com adversarial antes/depois (JOINS, janelas, CTEs, BRIN/B-tree/GIN, RLS) + 06-planos-antes-depois.md
3-casos	3 gargalos reais: fila mt_jobs, sync CRM e dashboard
7-apresentacao	deck, script de demonstracao e relatorio HTML/DOCX/PDF

6. Métricas de sucesso

METRICA	ATUAL	META
Worker da fila mt_jobs (pick job)	1.8s (Seq Scan)	< 10ms
JOIN de sync CRM (auditoria 30d)	4.2s	< 250ms
Dashboard de performance (janela 7d)	8.4s	< 1.5s
Bloat em tabelas de log	nao monitorado	< 20%

Timeout de webhook por query lenta

~3/dia

0

Nota: nenhum indice foi aplicado em producao nesta etapa: apenas documentado e provado.

7. Decisoões e tradeoffs

- **Nenhuma query em producao sem EXPLAIN (ANALYZE, BUFFERS):** exige disciplina de revisao e staging com volumetria representativa.
- **Indice composto na ordem igualdade, range e depois ORDER BY:** a escrita paga um pouco mais por indice; aceito porque a fila e lida pelo worker a cada 15s.
- **BRIN 10x menor que B-tree em created_at:** so vale se a ordem fisica acompanha o tempo; exige manutencao com vacuum e particionamento.
- **Particionamento por range com TTL em append-only:** exige job de manutencao; sem ele a tabela cresce sem limite.
- **CREATE INDEX CONCURRENTLY fora de pico:** criacao mais lenta e fora de transacao; a janela de deploy precisa prever isso.

8. Impacto no negocio

A fila `mt_jobs` com 2,4M linhas, o sync de CRM e o dashboard sobre 12M linhas geravam timeouts de webhook (cerca de 3 por dia), dashboards de 8s ou mais e backlog invisivel. Com os indices e o metodo EXPLAIN, o pick fica abaixo de 10ms, o sync abaixo de 250ms e o dashboard abaixo de 1,5s, zerando timeouts e devolvendo visibilidade da fila sem trocar de banco.

9. Referencias de estudo

- Curso: SQL Performance Explained, de Markus Winand (use-the-index-luke.com)
- Video: Postgres Performance (Supabase, YouTube)
- Doc oficial: Using EXPLAIN (PostgreSQL), postgresql.org/docs/current/using-explain.html (verificada em 2026-09-28)
- Doc oficial: Query Optimization (Supabase), supabase.com/docs/guides/database/query-optimization (verificada em 2026-09-28)

Status: Desenvolvido e em homologacao. Nenhum indice aplicado em producao nesta etapa. Proximos passos: rodar os planos no staging, aplicar indices com CONCURRENTLY, configurar job de

particionamento e validar RLS.